



Name: Cohort/Batch: Date: Score:

SERVERLESS PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Architecture

TOTAL: 10 QUESTIONS

Q1 What is Serverless and what problem in Architecture does it solve?

Q6 How does Serverless handle reliability and high availability?

Q2 What are the core components or concepts in Serverless?

Q7 What observability does Serverless provide out of the box?

Q3 How does Serverless compare to its main alternatives?

Q8 What are common performance bottlenecks in Serverless?

Q4 How do you get started with Serverless for a new project?

Q9 What security best practices apply when running Serverless?

Q5 What configuration options most affect Serverless's behavior in production?

Q10 What mistakes do teams commonly make when adopting Serverless?



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Serverless is a tool or platform that

Mitigation

Serverless is a tool or platform that automates or simplifies a specific concern in the Architecture domain, reducing manual effort and operational complexity for engineering teams.

A6 Serverless provides HA through leader election, replication, or stateless horizontal scaling.

Serverless provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A2 Serverless has key building blocks that work

Primitives

Serverless has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A7 Serverless exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces.

Serverless exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A3 Serverless trades off simplicity against feature richness

Hardening

Serverless trades off simplicity against feature richness differently than its alternatives. Choose Serverless when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A8 Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches.

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A4 Install Serverless via its package manager or binary release.

Gotchas

Install Serverless via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A9 Run Serverless with the principle of least privilege

Assessment

Run Serverless with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A5 Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels.

Configuration

Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A10 Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.